UNIVERSIDAD DON BOSCO

FACULTAD DE INGENIERÍA | ASB404

PANADERÍA BUEN PAN

Documento técnico y explicativo

INFORMACIÓN ACADÉMICA	DETALLE
Proyecto	Desafío práctico #3 - Panadería Buen Pan
Integrante 1	Kevin Isaac Figueroa Calderón
Integrante 2	[Completar nombre y carné]
Integrante 3	[Completar nombre y carné]
Asignatura	ASB404
Grupo / sección	[Completar]
Fecha de entrega	28 de octubre de 2026

Solución académica de base de datos relacional con automatización, consultas de negocio y control de acceso.

1. Resumen del proyecto

La panadería Buen Pan necesita administrar productos, clientes, empleados, ventas y existencias con una solución centralizada. Se diseñó una base de datos en Microsoft SQL Server con tablas relacionadas, reglas de integridad, reportes, procedimientos almacenados, automatización de inventario y acceso diferenciado por responsabilidades.

El sistema es una solución SQL de laboratorio; no incluye interfaz gráfica ni conexión a un servicio fiscal. El IVA se calcula al 13 % como supuesto académico aplicado a precios sin IVA.

2. Objetivos

- Construir un modelo relacional que reduzca duplicaciones y mantenga consistencia entre ventas, productos y personas.
- Automatizar ventas, existencias, movimientos y auditoría por medio de procedimientos y triggers.
- Ofrecer indicadores comerciales mediante al menos cinco vistas SQL.
- Separar las operaciones de caja, inventario y gerencia mediante roles y permisos.
- Demostrar resultados, controles de errores y escenarios verificables en SQL Server Management Studio (SSMS).

3. Entorno y archivos entregables

Elemento	Descripción
Motor de datos	SQL Server 2019/2022 o edición compatible; scripts T-SQL con GO y CREATE OR ALTER.
Herramienta de ejecución	SQL Server Management Studio (SSMS) o Azure Data Studio compatible con ejecución de lotes GO.
Instalación	SQL/00_Instalar_Todo.sql crea base, objetos, permisos y datos iniciales.
Demostración	SQL/07_PruebasGuiadas.sql consulta vistas, ejecuta operaciones, prueba validaciones y roles.
Scripts por módulo	SQL/01_Estructura.sql hasta SQL/06_DatosPrueba.sql.
Material gráfico	Diagrama/Diagrama_Entidad_Relacion.png y .pdf.

4. Arquitectura relacional

Modelo entidad-relación. PK identifica una fila y FK vincula entidades. Las cardinalidades 1:N muestran un registro padre vinculado a múltiples registros hijos.

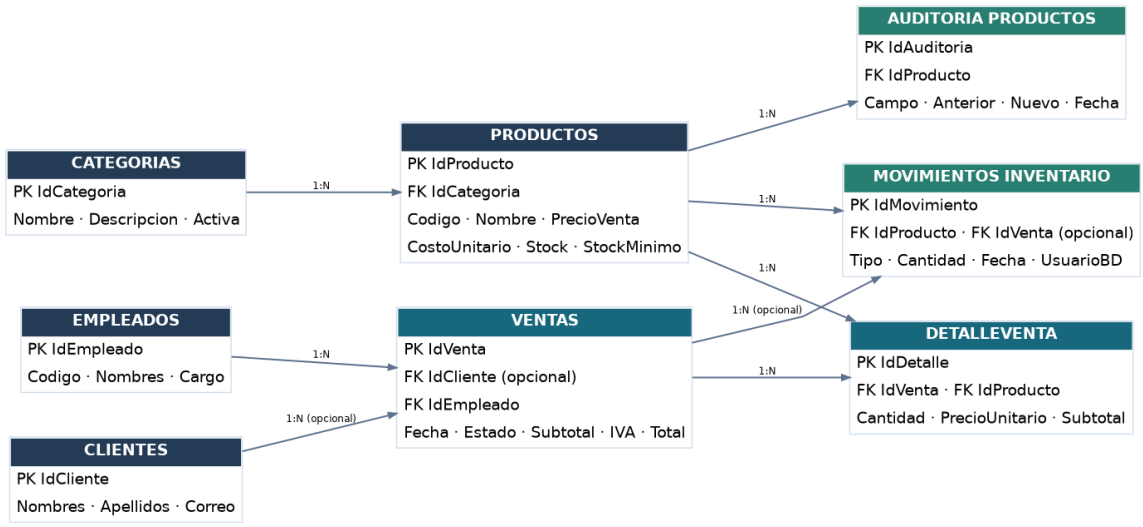


Figura 1. Esquema lógico de las ocho tablas.

5. Tablas y función de cada una

Tabla	Función y reglas
Categorias	Clasifica los productos mediante un identificador único y nombre irrepetible.
Productos	Registra código, categoría, precio, costo, stock, mínimo y estado activo; prohíbe existencias negativas.
Clientes	Contiene nombres, contactos y correo no duplicado cuando se informa.
Empleados	Identifica personal, cargo y disponibilidad para registrar operaciones.
Ventas	Encabezado de la compra; referencia al empleado y opcionalmente al cliente, guarda importes y estado.
DetalleVenta	Cada línea posee producto, cantidad, precio al momento de vender y subtotal calculado.
MovimientosInventario	Historial de entradas, salidas y devoluciones con usuario y venta cuando corresponde.
AuditoriaProductos	Almacena valor anterior y posterior de stock, precio, costo y actividad.

Normalización: categorías, personas, productos, ventas y detalle se almacenan separadamente. Los productos tienen su categoría por clave foránea; cada línea registra una sola referencia de producto; las descripciones no se repiten en cada venta. Los totales conservados en Ventas son una instantánea comercial intencional y no reemplazan el detalle que los origina.

6. Reglas de integridad

- Claves primarias, foráneas y restricciones UNIQUE protegen identidad, relaciones y duplicados.
- CHECK verifica cantidades positivas, precios válidos, inventario no negativo, estados permitidos y totales coherentes.
- Los importes se manejan con DECIMAL, evitando errores de representación típicos de FLOAT.

- Las transacciones con TRY-CATCH y XACT_ABORT cancelan una venta completa si falla alguno de sus pasos.
- Las ventas confirmadas y anuladas no admiten nuevas líneas; los artículos del detalle no pueden editarse ni borrarse directamente.

7. Vistas de información (8)

Vista	Qué responde
vw_CatalogoProductos	Precios, categoría, stock y disponibilidad del catálogo.
vw_ResumenVentasDiarias	Ventas por fecha, subtotal, impuesto, total y ticket promedio.
vw_DetalleVentas	Desglose de venta, cliente, empleado y producto.
vw_StockCritico	Productos con existencias iguales o inferiores al mínimo.
vw_TopProductos	Unidades vendidas y monto neto por producto en ventas confirmadas.
vw_ClientesFrecuentes	Compras acumuladas y promedio por cliente registrado.
vw_MovimientosInventario	Entradas, salidas y devoluciones ordenables por fecha.
vw_RentabilidadProductos	Costo, precio y margen unitario/porcentual de cada producto.

8. Procedimientos almacenados (7)

Procedimiento	Operación
usp_RegistrarVenta	Abre venta en borrador, inserta primer producto y devuelve @IdVenta como parámetro OUTPUT.
usp_AgregarProductoVenta	Agrega una línea a una venta no cerrada y exige que el producto no esté repetido.
usp_ConfirmarVenta	Suma detalles, calcula IVA 13 %, fija importes y cambia estado a CONFIRMADA.
usp_RegistrarEntradaInventario	Incrementa existencias, actualiza costo opcional y registra el movimiento.
usp_AnularVenta	Devuelve existencias y conserva la venta como ANULADA para trazabilidad.
usp_ActualizarPrecioProducto	Modifica precio validando valores positivos; el trigger audita.
usp_RegistrarCliente	Valida nombres y correo, inserta persona y devuelve @IdCliente OUTPUT.

9. Triggers (4)

Trigger	Efecto automático
trg_DetalleVenta_DescontarInventario	Cuando INSERT agrega líneas, verifica venta/producto y descuenta unidades agrupadas; rechaza stock insuficiente; registra SALIDA.
trg_DetalleVenta_BloquearEdicion	Rechaza UPDATE y DELETE del detalle para conservar el movimiento original.
trg_Productos_Auditoria	Ante UPDATE registra campo, valor anterior/nuevo, usuario y fecha si cambian stock, precio, costo o estado.

trg_Ventas_ValidarEstados	Prohíbe transiciones inválidas, cambiar importes de ventas cerradas o confirmar una venta vacía.

Ejemplo: vender 10 panes franceses disminuye el stock de 160 a 150. Una venta anulada devuelve sus existencias y registra la devolución sin borrar los movimientos previos.

10. Seguridad y responsabilidades

Rol	Permisos permitidos
rol_cajero	Registrar ventas, añadir productos, confirmar y registrar clientes; consultar catálogo y detalle.
rol_inventario	Registrar entradas, modificar precios y consultar catálogo, stock crítico y movimientos.
rol_gerencia	Todos los procedimientos, incluidas anulaciones, y todas las vistas de reportes.

La demostración crea demo_cajero, demo_inventario y demo_gerencia como usuarios de base de datos SIN LOGIN para ejecutar pruebas con EXECUTE AS USER. Esto no configura credenciales de acceso externo: para un despliegue real sería necesario crear identidades corporativas y asignarlas a los roles.

La separación de permisos se realiza sobre vistas y procedimientos mediante GRANT. No se otorgan privilegios directos de modificación sobre tablas a los tres roles; el acceso controlado opera mediante procedimientos con el mismo propietario de las tablas.

11. Casos de uso y datos de prueba

Escenario inicial cargado	Resultado esperado
Registrar 10 panes franceses y 2 cafés; confirmar.	Subtotal $4.20, IVA $0.55, total $4.75.
Registrar 2 porciones de pastel y 4 galletas; confirmar.	Subtotal $8.30, IVA $1.08, total $9.38.
Registrar 6 panes dulces y 1 quesadilla; confirmar.	Subtotal $4.20, IVA $0.55, total $4.75.
Crear venta de 2 chocolates y anular.	Stock de chocolate vuelve al valor anterior; estado ANULADA.
Ingresar 12 quesadillas al inventario.	Stock final inicial de quesadilla: 19.
Intentar vender 999 999 panes.	Operación rechazada; inventario no cambia.

Tras ejecutar solamente el instalador inicial, el total de las tres ventas CONFIRMADAS es $18.88. La venta ANULADA no se considera para reportes comerciales. Al ejecutar 07_PruebasGuiadas.sql se generan más operaciones, por lo que cambian los indicadores.

12. Protocolo de instalación y demostración

1. Iniciar SQL Server y abrir SSMS con una conexión que tenga permiso para crear bases de datos y objetos.

2. Abrir SQL/00_Instalar_Todo.sql y presionar Ejecutar; usar una instancia sin la base BuenPan_ASB404.

3. Confirmar que el panel Objetos contiene BuenPan_ASB404 y expandir Tablas, Vistas, Programabilidad y Seguridad.

4. Revisar la salida de conteos del instalador: 4 categorías, 10 productos, 6 clientes, 4 empleados y 4 ventas.

5. Ejecutar SQL/07_PruebasGuiadas.sql para obtener consultas, crear una nueva venta, probar una entrada y verificar seguridad.

6. Ejecutar bloques por separado durante la grabación para mantener visibles los resultados de cada caso.

7. Guardar capturas REALES de SSMS y reemplazar los espacios indicados en la sección de evidencias del PDF.

13. Evidencias a capturar en SSMS

Captura	Ventana/resultado necesario
E1 - Estructura	Explorador de objetos con base y sus 8 tablas.
E2 - Diagrama	Diagrama entidad-relación incluido en el proyecto.
E3 - Vistas	Resultado de vw_ResumenVentasDiarias y vw_StockCritico.
E4 - Procedimientos	Venta registrada, detalles y total calculado por usp_ConfirmarVenta.
E5 - Automatización	Stock actualizado, movimientos SALIDA/ENTRADA y registros de AuditoriaProductos.
E6 - Seguridad	Resultados de HAS_PERMS_BY_NAME para roles simulados.
E7 - Manejo de errores	Mensaje de venta rechazada por inventario insuficiente.

Estado de verificación: los scripts y archivos se prepararon y revisaron estáticamente. No se ha ejecutado SQL Server en este entorno, por lo que no se presentan capturas ni resultados de ejecución como si fueran reales. La última evidencia debe obtenerse en SSMS.

14. Pruebas de calidad y límites

- La venta se ejecuta dentro de una transacción. Ante error de stock, THROW activa rollback y evita registros parciales.
- Se conserva trazabilidad con tablas de movimientos y auditoría; la anulación se efectúa por estado, no por eliminación.
- El detalle es inmutable después de su inserción; para corregir una venta debe anularse y registrarse una nueva.
- El stock es un conteo de unidades de producto terminado; no se modelan materias primas, recetas ni compras a proveedores.
- El cálculo del IVA es simplificado para fines académicos, no acredita cumplimiento fiscal ni emisión de documentos tributarios.

15. Conclusiones

El proyecto integra diseño normalizado, consultas analíticas, reglas automáticas y controles de acceso en una solución coherente para la panadería. La combinación de claves, CHECK, transacciones y triggers protege las operaciones de caja frente a inconsistencias habituales, mientras que las vistas transforman los registros en información útil para administrar ventas e inventario.

La estructura modular de scripts facilita la defensa y el mantenimiento. Como evolución se propone una interfaz para caja, autenticación vinculada a empleados y procesos de compra/proveedores, sin sustituir los controles implementados dentro de la base de datos.

16. Anexo de código íntegro

La edición PDF de este documento incorpora el contenido completo, sin comentarios, de 01_Estructura.sql, 02_Triggers.sql, 03_Procedimientos.sql, 04_Vistas.sql, 05_Seguridad.sql y 06_DatosPrueba.sql. Los scripts separados y el instalador equivalente están incluidos en la carpeta SQL del ZIP.

ANEXO DE EVIDENCIAS - SSMS

Completar con capturas propias al ejecutar los scripts en SQL Server. No se han inventado resultados de ejecución.

E1 - Estructura

Explorador con la base y sus 8 tablas

Insertar aquí una captura real de SQL Server Management Studio

E3 - Reportes

Resultados de ventas diarias y stock crítico

Insertar aquí una captura real de SQL Server Management Studio

E4 - Ventas

ID, líneas y montos al confirmar una venta

Insertar aquí una captura real de SQL Server Management Studio

ANEXO DE EVIDENCIAS - SSMS

Completar con capturas propias al ejecutar los scripts en SQL Server. No se han inventado resultados de ejecución.

E5 - Triggers

Movimientos y auditoría tras operaciones

Insertar aquí una captura real de SQL Server Management Studio

E6 - Roles

Matriz de permisos de tres usuarios

Insertar aquí una captura real de SQL Server Management Studio

E7 - Errores

Rechazo por falta de inventario

Insertar aquí una captura real de SQL Server Management Studio

ANEXO - SCRIPT SQL COMPLETO

Código 1

01_Estructura.sql

```
IF DB_ID(N'BuenPan_ASB404') IS NOT NULL
    THROW 51000, N'La base BuenPan_ASB404 ya existe. Ejecuta este instalador solo en un entorno nuevo.', 1;
GO
CREATE DATABASE BuenPan_ASB404;
GO
USE BuenPan_ASB404;
GO
CREATE TABLE dbo.Categorias
(
    IdCategoria INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Categorias PRIMARY KEY,
    Nombre NVARCHAR(80) NOT NULL CONSTRAINT UQ_Categorias_Nombre UNIQUE,
    Descripcion NVARCHAR(240) NULL,
    Activa BIT NOT NULL CONSTRAINT DF_Categorias_Activa DEFAULT (1),
    CONSTRAINT CK_Categorias_Nombre CHECK (LEN(LTRIM(RTRIM(Nombre))) > 0)
);
GO
CREATE TABLE dbo.Productos
(
    IdProducto INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Productos PRIMARY KEY,
    IdCategoria INT NOT NULL,
    Codigo NVARCHAR(20) NOT NULL CONSTRAINT UQ_Productos_Codigo UNIQUE,
    Nombre NVARCHAR(120) NOT NULL,
    PrecioVenta DECIMAL(12,2) NOT NULL,
    CostoUnitario DECIMAL(12,2) NOT NULL,
    Stock INT NOT NULL CONSTRAINT DF_Productos_Stock DEFAULT (0),
    StockMinimo INT NOT NULL CONSTRAINT DF_Productos_StockMinimo DEFAULT (5),
    Activo BIT NOT NULL CONSTRAINT DF_Productos_Activo DEFAULT (1),
    FechaCreacion DATETIME2(0) NOT NULL CONSTRAINT DF_Productos_Fecha DEFAULT (SYSDATETIME()),
    CONSTRAINT FK_Productos_Categorias FOREIGN KEY (IdCategoria) REFERENCES dbo.Categorias (IdCategoria),
    CONSTRAINT CK_Productos_Precio CHECK (PrecioVenta > 0),
    CONSTRAINT CK_Productos_Costo CHECK (CostoUnitario >= 0),
    CONSTRAINT CK_Productos_Stock CHECK (Stock >= 0),
    CONSTRAINT CK_Productos_StockMinimo CHECK (StockMinimo >= 0),
    CONSTRAINT CK_Productos_Nombre CHECK (LEN(LTRIM(RTRIM(Nombre))) > 0)
);
GO
CREATE TABLE dbo.Clientes
(
    IdCliente INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Clientes PRIMARY KEY,
    Nombres NVARCHAR(100) NOT NULL,
    Apellidos NVARCHAR(100) NOT NULL,
    Telefono NVARCHAR(20) NULL,
    Correo NVARCHAR(150) NULL,
    Activo BIT NOT NULL CONSTRAINT DF_Clientes_Activo DEFAULT (1),
    FechaRegistro DATETIME2(0) NOT NULL CONSTRAINT DF_Clientes_Fecha DEFAULT (SYSDATETIME()),
```

01_Estructura.sql (continuación)

```
    CONSTRAINT CK_Clientes_Nombres CHECK (LEN(LTRIM(RTRIM(Nombres))) > 0),
    CONSTRAINT CK_Clientes_Apellidos CHECK (LEN(LTRIM(RTRIM(Apellidos))) > 0),
    CONSTRAINT CK_Clientes_Correo CHECK (Correo IS NULL OR (Correo LIKE N'%_@_%._%' AND Correo NOT LIKE N'% %'))
);
GO
CREATE UNIQUE INDEX UX_Clientes_Correo ON dbo.Clientes(Correo) WHERE Correo IS NOT NULL;
GO
CREATE TABLE dbo.Empleados
(
    IdEmpleado INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Empleados PRIMARY KEY,
    Codigo NVARCHAR(15) NOT NULL CONSTRAINT UQ_Empleados_Codigo UNIQUE,
    Nombres NVARCHAR(100) NOT NULL,
    Apellidos NVARCHAR(100) NOT NULL,
    Cargo NVARCHAR(60) NOT NULL,
    Activo BIT NOT NULL CONSTRAINT DF_Empleados_Activo DEFAULT (1),
    CONSTRAINT CK_Empleados_Nombres CHECK (LEN(LTRIM(RTRIM(Nombres))) > 0),
    CONSTRAINT CK_Empleados_Apellidos CHECK (LEN(LTRIM(RTRIM(Apellidos))) > 0)
);
GO
CREATE TABLE dbo.Ventas
(
    IdVenta INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Ventas PRIMARY KEY,
    Fecha DATETIME2(0) NOT NULL CONSTRAINT DF_Ventas_Fecha DEFAULT (SYSDATETIME()),
    IdCliente INT NULL,
    IdEmpleado INT NOT NULL,
    Estado VARCHAR(12) NOT NULL CONSTRAINT DF_Ventas_Estado DEFAULT ('BORRADOR'),
    Subtotal DECIMAL(14,2) NOT NULL CONSTRAINT DF_Ventas_Subtotal DEFAULT (0),
    IVA DECIMAL(14,2) NOT NULL CONSTRAINT DF_Ventas_IVA DEFAULT (0),
    Total DECIMAL(14,2) NOT NULL CONSTRAINT DF_Ventas_Total DEFAULT (0),
    CONSTRAINT FK_Ventas_Clientes FOREIGN KEY (IdCliente) REFERENCES dbo.Clientes(IdCliente),
    CONSTRAINT FK_Ventas_Empleados FOREIGN KEY (IdEmpleado) REFERENCES dbo.Empleados(IdEmpleado),
    CONSTRAINT CK_Ventas_Estado CHECK (Estado IN ('BORRADOR','CONFIRMADA','ANULADA')),
    CONSTRAINT CK_Ventas_Montos CHECK (Subtotal >= 0 AND IVA >= 0 AND Total >= 0 AND Total = Subtotal + IVA)
);
GO
CREATE TABLE dbo.DetalleVenta
(
    IdDetalle INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_DetalleVenta PRIMARY KEY,
    IdVenta INT NOT NULL,
    IdProducto INT NOT NULL,
    Cantidad INT NOT NULL,
    PrecioUnitario DECIMAL(12,2) NOT NULL,
    Subtotal AS CONVERT(DECIMAL(14,2), Cantidad * PrecioUnitario) PERSISTED,
    CONSTRAINT FK_DetalleVenta_Ventas FOREIGN KEY (IdVenta) REFERENCES dbo.Ventas(IdVenta),
    CONSTRAINT FK_DetalleVenta_Productos FOREIGN KEY (IdProducto) REFERENCES dbo.Productos(IdProducto),
```

01_Estructura.sql (continuación)

```
    CONSTRAINT UQ_DetalleVenta_VentaProducto UNIQUE (IdVenta, IdProducto),
    CONSTRAINT CK_DetalleVenta_Cantidad CHECK (Cantidad > 0),
    CONSTRAINT CK_DetalleVenta_Precio CHECK (PrecioUnitario > 0)
);
GO
CREATE TABLE dbo.MovimientosInventario
(
    IdMovimiento BIGINT IDENTITY(1,1) NOT NULL CONSTRAINT PK_MovimientosInventario PRIMARY KEY,
    IdProducto INT NOT NULL,
    IdVenta INT NULL,
    Tipo VARCHAR(12) NOT NULL,
    Cantidad INT NOT NULL,
    Fecha DATETIME2(0) NOT NULL CONSTRAINT DF_Movimientos_Fecha DEFAULT (SYSDATETIME()),
    UsuarioBD SYSNAME NOT NULL CONSTRAINT DF_Movimientos_Usuario DEFAULT (USER_NAME()),
    CONSTRAINT FK_Movimientos_Productos FOREIGN KEY (IdProducto) REFERENCES dbo.Productos(IdProducto),
    CONSTRAINT FK_Movimientos_Ventas FOREIGN KEY (IdVenta) REFERENCES dbo.Ventas(IdVenta),
    CONSTRAINT CK_Movimientos_Tipo CHECK (Tipo IN ('ENTRADA','SALIDA','DEVOLUCION')),
    CONSTRAINT CK_Movimientos_Cantidad CHECK (Cantidad > 0),
    CONSTRAINT CK_Movimientos_Venta CHECK ((Tipo = 'ENTRADA' AND IdVenta IS NULL) OR (Tipo IN ('SALIDA','DEVOLUCION') AND IdVenta IS NOT NULL))
);
GO
CREATE TABLE dbo.AuditoriaProductos
(
    IdAuditoria BIGINT IDENTITY(1,1) NOT NULL CONSTRAINT PK_AuditoriaProductos PRIMARY KEY,
    IdProducto INT NOT NULL,
    Campo NVARCHAR(35) NOT NULL,
    ValorAnterior NVARCHAR(100) NULL,
    ValorNuevo NVARCHAR(100) NULL,
    UsuarioBD SYSNAME NOT NULL,
    Fecha DATETIME2(0) NOT NULL CONSTRAINT DF_Auditoria_Fecha DEFAULT (SYSDATETIME()),
    CONSTRAINT FK_Auditoria_Productos FOREIGN KEY (IdProducto) REFERENCES dbo.Productos(IdProducto)
);
GO
CREATE INDEX IX_Productos_Categoria ON dbo.Productos(IdCategoria);
CREATE INDEX IX_Ventas_FechaEstado ON dbo.Ventas(Fecha, Estado) INCLUDE (Total, IdCliente, IdEmpleado);
CREATE INDEX IX_DetalleVenta_Producto ON dbo.DetalleVenta(IdProducto);
CREATE INDEX IX_Movimientos_ProductoFecha ON dbo.MovimientosInventario(IdProducto, Fecha);
CREATE INDEX IX_Auditoria_ProductoFecha ON dbo.AuditoriaProductos(IdProducto, Fecha);
GO
```

02_Triggers.sql

```
USE BuenPan_ASB404;
GO
CREATE OR ALTER TRIGGER dbo.trg_DetalleVenta_DescontarInventario
ON dbo.DetalleVenta
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS
    (
        SELECT 1
        FROM inserted AS i
        INNER JOIN dbo.Ventas AS v ON v.IdVenta = i.IdVenta
        WHERE v.Estado <> 'BORRADOR'
    )
        THROW 51001, N'Solo es posible agregar productos a ventas en borrador.', 1;

    IF EXISTS
    (
        SELECT 1
        FROM inserted AS i
        INNER JOIN dbo.Productos AS p ON p.IdProducto = i.IdProducto
        WHERE p.Activo = 0
    )
        THROW 51002, N'No es posible vender productos inactivos.', 1;

    ;WITH Consumo AS
    (
        SELECT IdProducto, SUM(Cantidad) AS Cantidad
        FROM inserted
        GROUP BY IdProducto
    )
    UPDATE p
       SET p.Stock = p.Stock - c.Cantidad
    FROM dbo.Productos AS p
    INNER JOIN Consumo AS c ON c.IdProducto = p.IdProducto
    WHERE p.Stock >= c.Cantidad AND p.Activo = 1;

    IF @@ROWCOUNT <> (SELECT COUNT(DISTINCT IdProducto) FROM inserted)
        THROW 51003, N'Inventario insuficiente para completar la operación.', 1;

    INSERT INTO dbo.MovimientosInventario(IdProducto, IdVenta, Tipo, Cantidad, UsuarioBD)
    SELECT IdProducto, IdVenta, 'SALIDA', SUM(Cantidad), USER_NAME()
    FROM inserted
```

02_Triggers.sql (continuación)

```
    GROUP BY IdProducto, IdVenta;
END;
GO
CREATE OR ALTER TRIGGER dbo.trg_DetalleVenta_BloquearEdicion
ON dbo.DetalleVenta
AFTER UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM deleted)
        THROW 51004, N'Las líneas de venta no pueden modificarse ni borrarse; anula la venta para devolver existencias.', 1;
END;
GO
CREATE OR ALTER TRIGGER dbo.trg_Productos_Auditoria
ON dbo.Productos
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.AuditoriaProductos
    (
        IdProducto, Campo, ValorAnterior, ValorNuevo, UsuarioBD
    )
    SELECT i.IdProducto, cambios.Campo, cambios.Anterior, cambios.Nuevo, USER_NAME()
    FROM inserted AS i
    INNER JOIN deleted AS d ON d.IdProducto = i.IdProducto
    CROSS APPLY
    (
        VALUES
        (N'Stock', CONVERT(NVARCHAR(100), d.Stock), CONVERT(NVARCHAR(100), i.Stock)),
        (N'PrecioVenta', CONVERT(NVARCHAR(100), d.PrecioVenta), CONVERT(NVARCHAR(100), i.PrecioVenta)),
        (N'CostoUnitario', CONVERT(NVARCHAR(100), d.CostoUnitario), CONVERT(NVARCHAR(100), i.CostoUnitario)),
        (N'Activo', CONVERT(NVARCHAR(100), d.Activo), CONVERT(NVARCHAR(100), i.Activo))
    ) AS cambios(Campo, Anterior, Nuevo)
    WHERE cambios.Anterior <> cambios.Nuevo;
END;
GO
CREATE OR ALTER TRIGGER dbo.trg_Ventas_ValidarEstados
ON dbo.Ventas
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
```

02_Triggers.sql (continuación)

```
    IF EXISTS
    (
        SELECT 1
        FROM inserted AS i
        INNER JOIN deleted AS d ON d.IdVenta = i.IdVenta
        WHERE i.Estado <> d.Estado
          AND NOT
          (
              (d.Estado = 'BORRADOR' AND i.Estado IN ('CONFIRMADA', 'ANULADA'))
              OR (d.Estado = 'CONFIRMADA' AND i.Estado = 'ANULADA')
          )
    )
        THROW 51005, N'Transición de estado no permitida.', 1;

    IF EXISTS
    (
        SELECT 1
        FROM inserted AS i
        INNER JOIN deleted AS d ON d.IdVenta = i.IdVenta
        WHERE d.Estado <> 'BORRADOR'
          AND (i.Subtotal <> d.Subtotal OR i.IVA <> d.IVA OR i.Total <> d.Total)
    )
        THROW 51006, N'No se pueden modificar los importes de ventas cerradas.', 1;

    IF EXISTS
    (
        SELECT 1
        FROM inserted AS i
        INNER JOIN deleted AS d ON d.IdVenta = i.IdVenta
        WHERE i.Estado = 'CONFIRMADA'
          AND d.Estado = 'BORRADOR'
          AND NOT EXISTS
          (
              SELECT 1 FROM dbo.DetalleVenta AS dv WHERE dv.IdVenta = i.IdVenta
          )
    )
        THROW 51007, N'No se puede confirmar una venta sin productos.', 1;
END;
GO
```

03_Procedimientos.sql

```
USE BuenPan_ASB404;
GO
CREATE OR ALTER PROCEDURE dbo.usp_RegistrarVenta
    @IdCliente INT = NULL,
    @IdEmpleado INT,
    @IdProducto INT,
    @Cantidad INT,
    @IdVenta INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        IF @Cantidad IS NULL OR @Cantidad <= 0
            THROW 51101, N'La cantidad debe ser mayor que cero.', 1;

        BEGIN TRANSACTION;

        IF NOT EXISTS (SELECT 1 FROM dbo.Empleados WHERE IdEmpleado = @IdEmpleado AND Activo = 1)
            THROW 51102, N'El empleado no existe o está inactivo.', 1;

        IF @IdCliente IS NOT NULL AND NOT EXISTS
        (
            SELECT 1 FROM dbo.Clientes WHERE IdCliente = @IdCliente AND Activo = 1
        )
            THROW 51103, N'El cliente no existe o está inactivo.', 1;

        DECLARE @Precio DECIMAL(12,2);
        SELECT @Precio = PrecioVenta FROM dbo.Productos WHERE IdProducto = @IdProducto AND Activo = 1;

        IF @Precio IS NULL
            THROW 51104, N'El producto no existe o está inactivo.', 1;

        INSERT INTO dbo.Ventas(IdCliente, IdEmpleado)
        VALUES (@IdCliente, @IdEmpleado);

        SET @IdVenta = CONVERT(INT, SCOPE_IDENTITY());

        INSERT INTO dbo.DetalleVenta(IdVenta, IdProducto, Cantidad, PrecioUnitario)
        VALUES (@IdVenta, @IdProducto, @Cantidad, @Precio);

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
```

03_Procedimientos.sql (continuación)

```
        IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
CREATE OR ALTER PROCEDURE dbo.usp_AgregarProductoVenta
    @IdVenta INT,
    @IdProducto INT,
    @Cantidad INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        IF @Cantidad IS NULL OR @Cantidad <= 0
            THROW 51105, N'La cantidad debe ser mayor que cero.', 1;

        BEGIN TRANSACTION;

        IF NOT EXISTS
        (
            SELECT 1 FROM dbo.Ventas WITH (UPDLOCK, HOLDLOCK)
            WHERE IdVenta = @IdVenta AND Estado = 'BORRADOR'
        )
            THROW 51106, N'La venta no existe o ya está cerrada.', 1;

        IF EXISTS
        (
            SELECT 1 FROM dbo.DetalleVenta
            WHERE IdVenta = @IdVenta AND IdProducto = @IdProducto
        )
            THROW 51107, N'El producto ya está en esta venta.', 1;

        DECLARE @Precio DECIMAL(12,2);
        SELECT @Precio = PrecioVenta FROM dbo.Productos WHERE IdProducto = @IdProducto AND Activo = 1;

        IF @Precio IS NULL
            THROW 51108, N'El producto no existe o está inactivo.', 1;

        INSERT INTO dbo.DetalleVenta(IdVenta, IdProducto, Cantidad, PrecioUnitario)
        VALUES (@IdVenta, @IdProducto, @Cantidad, @Precio);

        COMMIT TRANSACTION;
    END TRY
```

03_Procedimientos.sql (continuación)

```
    BEGIN CATCH
        IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
CREATE OR ALTER PROCEDURE dbo.usp_ConfirmarVenta
    @IdVenta INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        IF NOT EXISTS
        (
            SELECT 1 FROM dbo.Ventas WITH (UPDLOCK, HOLDLOCK)
            WHERE IdVenta = @IdVenta AND Estado = 'BORRADOR'
        )
            THROW 51109, N'La venta no existe o no está en borrador.', 1;

        DECLARE @Subtotal DECIMAL(14,2);
        DECLARE @IVA DECIMAL(14,2);

        SELECT @Subtotal = SUM(Subtotal)
        FROM dbo.DetalleVenta
        WHERE IdVenta = @IdVenta;

        IF @Subtotal IS NULL OR @Subtotal <= 0
            THROW 51110, N'No se puede confirmar una venta sin detalle.', 1;

        SET @IVA = CONVERT(DECIMAL(14,2), ROUND(@Subtotal * 0.13, 2));

        UPDATE dbo.Ventas
           SET Subtotal = @Subtotal,
               IVA = @IVA,
               Total = @Subtotal + @IVA,
               Estado = 'CONFIRMADA'
        WHERE IdVenta = @IdVenta;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
```

03_Procedimientos.sql (continuación)

```
        IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
CREATE OR ALTER PROCEDURE dbo.usp_RegistrarEntradaInventario
    @IdProducto INT,
    @Cantidad INT,
    @CostoUnitario DECIMAL(12,2) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        IF @Cantidad IS NULL OR @Cantidad <= 0
            THROW 51111, N'La entrada debe ser mayor que cero.', 1;
        IF @CostoUnitario IS NOT NULL AND @CostoUnitario < 0
            THROW 51112, N'El costo no puede ser negativo.', 1;

        BEGIN TRANSACTION;

        UPDATE dbo.Productos
           SET Stock = Stock + @Cantidad,
               CostoUnitario = COALESCE(@CostoUnitario, CostoUnitario)
        WHERE IdProducto = @IdProducto AND Activo = 1;

        IF @@ROWCOUNT <> 1
            THROW 51113, N'No se encontró un producto activo para ingresar inventario.', 1;

        INSERT INTO dbo.MovimientosInventario(IdProducto, Tipo, Cantidad, UsuarioBD)
        VALUES (@IdProducto, 'ENTRADA', @Cantidad, USER_NAME());

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
CREATE OR ALTER PROCEDURE dbo.usp_AnularVenta
    @IdVenta INT
AS
BEGIN
```

03_Procedimientos.sql (continuación)

```
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        IF NOT EXISTS
        (
            SELECT 1 FROM dbo.Ventas WITH (UPDLOCK, HOLDLOCK)
            WHERE IdVenta = @IdVenta AND Estado IN ('BORRADOR','CONFIRMADA')
        )
            THROW 51114, N'La venta no existe o ya fue anulada.', 1;

        ;WITH Devolucion AS
        (
            SELECT IdProducto, SUM(Cantidad) AS Cantidad
            FROM dbo.DetalleVenta
            WHERE IdVenta = @IdVenta
            GROUP BY IdProducto
        )
        UPDATE p
           SET p.Stock = p.Stock + d.Cantidad
        FROM dbo.Productos AS p
        INNER JOIN Devolucion AS d ON d.IdProducto = p.IdProducto;

        INSERT INTO dbo.MovimientosInventario(IdProducto, IdVenta, Tipo, Cantidad, UsuarioBD)
        SELECT IdProducto, @IdVenta, 'DEVOLUCION', SUM(Cantidad), USER_NAME()
        FROM dbo.DetalleVenta
        WHERE IdVenta = @IdVenta
        GROUP BY IdProducto;

        UPDATE dbo.Ventas SET Estado = 'ANULADA' WHERE IdVenta = @IdVenta;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
GO
CREATE OR ALTER PROCEDURE dbo.usp_ActualizarPrecioProducto
    @IdProducto INT,
    @NuevoPrecio DECIMAL(12,2)
AS
```

03_Procedimientos.sql (continuación)

```
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    IF @NuevoPrecio IS NULL OR @NuevoPrecio <= 0
        THROW 51115, N'El precio debe ser mayor que cero.', 1;

    UPDATE dbo.Productos
       SET PrecioVenta = @NuevoPrecio
    WHERE IdProducto = @IdProducto AND Activo = 1;

    IF @@ROWCOUNT <> 1
        THROW 51116, N'No existe el producto activo.', 1;
END;
GO
CREATE OR ALTER PROCEDURE dbo.usp_RegistrarCliente
    @Nombres NVARCHAR(100),
    @Apellidos NVARCHAR(100),
    @Telefono NVARCHAR(20) = NULL,
    @Correo NVARCHAR(150) = NULL,
    @IdCliente INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    IF NULLIF(LTRIM(RTRIM(@Nombres)), N'') IS NULL OR NULLIF(LTRIM(RTRIM(@Apellidos)), N'') IS NULL
        THROW 51117, N'Debes ingresar nombres y apellidos.', 1;

    IF @Correo IS NOT NULL AND (@Correo NOT LIKE N'%_@_%._%' OR @Correo LIKE N'% %')
        THROW 51118, N'El formato del correo no es válido.', 1;

    INSERT INTO dbo.Clientes(Nombres, Apellidos, Telefono, Correo)
    VALUES (LTRIM(RTRIM(@Nombres)), LTRIM(RTRIM(@Apellidos)), @Telefono, NULLIF(LTRIM(RTRIM(@Correo)), N''));

    SET @IdCliente = CONVERT(INT, SCOPE_IDENTITY());
END;
GO
```

04_Vistas.sql

```
USE BuenPan_ASB404;
GO
CREATE OR ALTER VIEW dbo.vw_CatalogoProductos
AS
SELECT p.IdProducto, p.Codigo, p.Nombre AS Producto, c.Nombre AS Categoria,
       p.PrecioVenta, p.Stock, p.StockMinimo,
       CASE WHEN p.Stock <= p.StockMinimo THEN N'REPOSICIÓN' ELSE N'DISPONIBLE' END AS EstadoInventario
FROM dbo.Productos AS p
INNER JOIN dbo.Categorias AS c ON c.IdCategoria = p.IdCategoria
WHERE p.Activo = 1;
GO
CREATE OR ALTER VIEW dbo.vw_ResumenVentasDiarias
AS
SELECT CONVERT(DATE, Fecha) AS Fecha,
       COUNT(*) AS NumeroVentas,
       SUM(Subtotal) AS Subtotal,
       SUM(IVA) AS IVA,
       SUM(Total) AS Total,
       CONVERT(DECIMAL(14,2), AVG(Total)) AS TicketPromedio
FROM dbo.Ventas
WHERE Estado = 'CONFIRMADA'
GROUP BY CONVERT(DATE, Fecha);
GO
CREATE OR ALTER VIEW dbo.vw_DetalleVentas
AS
SELECT v.IdVenta, v.Fecha, v.Estado,
       COALESCE(c.Nombres + N' ' + c.Apellidos, N'Consumidor final') AS Cliente,
       e.Nombres + N' ' + e.Apellidos AS Empleado,
       p.Codigo, p.Nombre AS Producto,
       d.Cantidad, d.PrecioUnitario, d.Subtotal AS TotalLinea,
       v.Total AS TotalVenta
FROM dbo.DetalleVenta AS d
INNER JOIN dbo.Ventas AS v ON v.IdVenta = d.IdVenta
INNER JOIN dbo.Empleados AS e ON e.IdEmpleado = v.IdEmpleado
LEFT JOIN dbo.Clientes AS c ON c.IdCliente = v.IdCliente
INNER JOIN dbo.Productos AS p ON p.IdProducto = d.IdProducto;
GO
CREATE OR ALTER VIEW dbo.vw_StockCritico
AS
SELECT p.IdProducto, p.Codigo, p.Nombre AS Producto, c.Nombre AS Categoria,
       p.Stock, p.StockMinimo,
       p.StockMinimo - p.Stock AS UnidadesPorReponer
FROM dbo.Productos AS p
INNER JOIN dbo.Categorias AS c ON c.IdCategoria = p.IdCategoria
WHERE p.Activo = 1 AND p.Stock <= p.StockMinimo;
```

04_Vistas.sql (continuación)

```
GO
CREATE OR ALTER VIEW dbo.vw_TopProductos
AS
SELECT p.IdProducto, p.Codigo, p.Nombre AS Producto,
       SUM(d.Cantidad) AS UnidadesVendidas,
       SUM(d.Subtotal) AS VentaSinIVA,
       COUNT(DISTINCT v.IdVenta) AS VentasDistintas
FROM dbo.DetalleVenta AS d
INNER JOIN dbo.Ventas AS v ON v.IdVenta = d.IdVenta
INNER JOIN dbo.Productos AS p ON p.IdProducto = d.IdProducto
WHERE v.Estado = 'CONFIRMADA'
GROUP BY p.IdProducto, p.Codigo, p.Nombre;
GO
CREATE OR ALTER VIEW dbo.vw_ClientesFrecuentes
AS
SELECT c.IdCliente, c.Nombres + N' ' + c.Apellidos AS Cliente,
       COUNT(v.IdVenta) AS ComprasRealizadas,
       SUM(v.Total) AS TotalComprado,
       CONVERT(DECIMAL(14,2), AVG(v.Total)) AS CompraPromedio
FROM dbo.Clientes AS c
INNER JOIN dbo.Ventas AS v ON v.IdCliente = c.IdCliente
WHERE v.Estado = 'CONFIRMADA'
GROUP BY c.IdCliente, c.Nombres, c.Apellidos;
GO
CREATE OR ALTER VIEW dbo.vw_MovimientosInventario
AS
SELECT m.IdMovimiento, m.Fecha, p.Codigo, p.Nombre AS Producto,
       m.Tipo, m.Cantidad, m.IdVenta, m.UsuarioBD
FROM dbo.MovimientosInventario AS m
INNER JOIN dbo.Productos AS p ON p.IdProducto = m.IdProducto;
GO
CREATE OR ALTER VIEW dbo.vw_RentabilidadProductos
AS
SELECT p.Codigo, p.Nombre AS Producto, p.CostoUnitario, p.PrecioVenta,
       p.PrecioVenta - p.CostoUnitario AS MargenUnitario,
       CONVERT(DECIMAL(7,2), (p.PrecioVenta - p.CostoUnitario) * 100.0 / p.PrecioVenta) AS MargenPorcentaje
FROM dbo.Productos AS p
WHERE p.Activo = 1;
GO
```

05_Seguridad.sql

```
USE BuenPan_ASB404;
GO
CREATE ROLE rol_cajero AUTHORIZATION dbo;
CREATE ROLE rol_inventario AUTHORIZATION dbo;
CREATE ROLE rol_gerencia AUTHORIZATION dbo;
GO
GRANT EXECUTE ON OBJECT::dbo.usp_RegistrarVenta TO rol_cajero;
GRANT EXECUTE ON OBJECT::dbo.usp_AgregarProductoVenta TO rol_cajero;
GRANT EXECUTE ON OBJECT::dbo.usp_ConfirmarVenta TO rol_cajero;
GRANT EXECUTE ON OBJECT::dbo.usp_RegistrarCliente TO rol_cajero;
GRANT SELECT ON OBJECT::dbo.vw_CatalogoProductos TO rol_cajero;
GRANT SELECT ON OBJECT::dbo.vw_DetalleVentas TO rol_cajero;
GO
GRANT EXECUTE ON OBJECT::dbo.usp_RegistrarEntradaInventario TO rol_inventario;
GRANT EXECUTE ON OBJECT::dbo.usp_ActualizarPrecioProducto TO rol_inventario;
GRANT SELECT ON OBJECT::dbo.vw_CatalogoProductos TO rol_inventario;
GRANT SELECT ON OBJECT::dbo.vw_StockCritico TO rol_inventario;
GRANT SELECT ON OBJECT::dbo.vw_MovimientosInventario TO rol_inventario;
GO
GRANT EXECUTE ON OBJECT::dbo.usp_RegistrarVenta TO rol_gerencia;
GRANT EXECUTE ON OBJECT::dbo.usp_AgregarProductoVenta TO rol_gerencia;
GRANT EXECUTE ON OBJECT::dbo.usp_ConfirmarVenta TO rol_gerencia;
GRANT EXECUTE ON OBJECT::dbo.usp_RegistrarEntradaInventario TO rol_gerencia;
GRANT EXECUTE ON OBJECT::dbo.usp_AnularVenta TO rol_gerencia;
GRANT EXECUTE ON OBJECT::dbo.usp_ActualizarPrecioProducto TO rol_gerencia;
GRANT EXECUTE ON OBJECT::dbo.usp_RegistrarCliente TO rol_gerencia;
GRANT SELECT ON OBJECT::dbo.vw_CatalogoProductos TO rol_gerencia;
GRANT SELECT ON OBJECT::dbo.vw_ResumenVentasDiarias TO rol_gerencia;
GRANT SELECT ON OBJECT::dbo.vw_DetalleVentas TO rol_gerencia;
GRANT SELECT ON OBJECT::dbo.vw_StockCritico TO rol_gerencia;
GRANT SELECT ON OBJECT::dbo.vw_TopProductos TO rol_gerencia;
GRANT SELECT ON OBJECT::dbo.vw_ClientesFrecuentes TO rol_gerencia;
GRANT SELECT ON OBJECT::dbo.vw_MovimientosInventario TO rol_gerencia;
GRANT SELECT ON OBJECT::dbo.vw_RentabilidadProductos TO rol_gerencia;
GO
CREATE USER demo_cajero WITHOUT LOGIN;
CREATE USER demo_inventario WITHOUT LOGIN;
CREATE USER demo_gerencia WITHOUT LOGIN;
GO
ALTER ROLE rol_cajero ADD MEMBER demo_cajero;
ALTER ROLE rol_inventario ADD MEMBER demo_inventario;
ALTER ROLE rol_gerencia ADD MEMBER demo_gerencia;
GO
```

06_DatosPrueba.sql

```
USE BuenPan_ASB404;
GO
INSERT INTO dbo.Categorias (Nombre, Descripcion)
VALUES (N'Panes', N'Panes artesanales y tradicionales'),
       (N'Repostería', N'Postres frescos y pasteles'),
       (N'Bebidas', N'Bebidas calientes y frías'),
       (N'Galletas', N'Galletas de elaboración diaria');
GO
INSERT INTO dbo.Productos (IdCategoria, Codigo, Nombre, PrecioVenta, CostoUnitario, Stock, StockMinimo)
VALUES (1,N'PAN001',N'Pan francés',0.20,0.10,160,20),
       (1,N'PAN002',N'Pan dulce surtido',0.45,0.22,90,15),
       (1,N'PAN003',N'Pan integral',1.30,0.75,25,8),
       (1,N'PAN004',N'Baguette artesanal',1.75,0.95,15,6),
       (2,N'REP001',N'Porción de pastel de chocolate',2.75,1.40,20,5),
       (2,N'REP002',N'Quesadilla salvadoreña',1.50,0.80,8,8),
       (3,N'BEB001',N'Café americano',1.10,0.40,70,10),
       (3,N'BEB002',N'Chocolate caliente',1.60,0.65,30,8),
       (4,N'GAL001',N'Galleta de avena',0.70,0.30,55,10),
       (4,N'GAL002',N'Galleta de mantequilla',0.60,0.24,45,10);
GO
INSERT INTO dbo.Clientes(Nombres, Apellidos, Telefono, Correo)
VALUES (N'Ana',N'Martínez',N'7010-1001',N'ana.martinez@example.com'),
       (N'Carlos',N'Pérez',N'7010-1002',N'carlos.perez@example.com'),
       (N'María',N'López',N'7010-1003',N'maria.lopez@example.com'),
       (N'José',N'Ramírez',N'7010-1004',N'jose.ramirez@example.com'),
       (N'Lucía',N'Hernández',N'7010-1005',N'lucia.hernandez@example.com'),
       (N'Daniel',N'Morales',N'7010-1006',N'daniel.morales@example.com');
GO
INSERT INTO dbo.Empleados(Codigo, Nombres, Apellidos, Cargo)
VALUES (N'EMP001',N'Sofía',N'Gómez',N'Cajera'),
       (N'EMP002',N'Luis',N'Rivera',N'Cajero'),
       (N'EMP003',N'Valeria',N'Torres',N'Encargada de inventario'),
       (N'EMP004',N'Óscar',N'Mendoza',N'Gerente');
GO
DECLARE @VentaA INT, @VentaB INT, @VentaC INT, @VentaD INT;
EXEC dbo.usp_RegistrarVenta @IdCliente=1, @IdEmpleado=1, @IdProducto=1, @Cantidad=10, @IdVenta=@VentaA OUTPUT;
EXEC dbo.usp_AgregarProductoVenta @IdVenta=@VentaA, @IdProducto=7, @Cantidad=2;
EXEC dbo.usp_ConfirmarVenta @IdVenta=@VentaA;
EXEC dbo.usp_RegistrarVenta @IdCliente=2, @IdEmpleado=2, @IdProducto=5, @Cantidad=2, @IdVenta=@VentaB OUTPUT;
EXEC dbo.usp_AgregarProductoVenta @IdVenta=@VentaB, @IdProducto=9, @Cantidad=4;
EXEC dbo.usp_ConfirmarVenta @IdVenta=@VentaB;
EXEC dbo.usp_RegistrarVenta @IdCliente=1, @IdEmpleado=1, @IdProducto=2, @Cantidad=6, @IdVenta=@VentaC OUTPUT;
EXEC dbo.usp_AgregarProductoVenta @IdVenta=@VentaC, @IdProducto=6, @Cantidad=1;
EXEC dbo.usp_ConfirmarVenta @IdVenta=@VentaC;
EXEC dbo.usp_RegistrarVenta @IdCliente=3, @IdEmpleado=2, @IdProducto=8, @Cantidad=2, @IdVenta=@VentaD OUTPUT;
```

06_DatosPrueba.sql (continuación)

```
EXEC dbo.usp_AnularVenta @IdVenta=@VentaD;
EXEC dbo.usp_RegistrarEntradaInventario @IdProducto=6, @Cantidad=12, @CostoUnitario=0.85;
GO
SELECT (SELECT COUNT(*) FROM dbo.Categorias) AS Categorias,
       (SELECT COUNT(*) FROM dbo.Productos) AS Productos,
       (SELECT COUNT(*) FROM dbo.Clientes) AS Clientes,
       (SELECT COUNT(*) FROM dbo.Empleados) AS Empleados,
       (SELECT COUNT(*) FROM dbo.Ventas) AS Ventas,
       (SELECT COUNT(*) FROM dbo.MovimientosInventario) AS Movimientos,
       (SELECT COUNT(*) FROM dbo.AuditoriaProductos) AS Auditorias;
GO
```